3. Next, let's create a new feature table from our DataFrame. We will assume that the maximum price for a specific product does not vary enough to calculate this value more than hourly, so we can set this table to update on a set schedule. In the GitHub code, we'll instantiate `FeatureEngineeringClient`. In *Figure 5.31*, we use it to write the new feature table as a Delta table in Unity Catalog:

```
fe.create_table(
  df=max_price_df,
  name='product_3minute_max_price_ft',
  primary_keys=['Product','LookupTimestamp'],
  timeseries_columns='LookupTimestamp',
  schema=max_price_df.schema,
  description="Maximum price per product over the last 3 minutes for Synthetic
  Transactions. Join on TransactionTimestamp to get the max product price from last
  minute's 3 minute rolling max"
)
```

Figure 5.31 – Writing our table to a Delta table in Unity Catalog

4. Next, we need a Python UDF to calculate the discount or the difference between the transaction amount and the product's maximum price. We'll name it `product_difference_ratio_on_demand_feature`. We can use the same notebook to build and save this simple function under the same catalog and schema as our tables in Unity Catalog.

```
%sql
CREATE FUNCTION IF NOT EXISTS product_difference_ratio_on_demand_feature(max_price FLOAT, transaction_amount FLOAT)
RETURNS float
LANGUAGE PYTHON
COMMENT 'Calculate the difference ratio for a product at time of transaction (maximum price - transaction amount)/maximum price.'
AS $$
def calc_ratio_difference(n1: float, n2: float) -> float:
  return round(((n1 - n2)/n1),2)

return calc_ratio_difference(max_price, transaction_amount)
$$
```

Figure 5.32 – Building an on-demand function to calculate product discounts